StatementGuard

PROTOTYPE

AI screening of corporate-loan bank statements for tampering - before a credit analyst ever sees them.

A multi-agent pipeline that auto-clears clean files, routes suspicious ones to a human, and attaches readable evidence to every flag. Built to MAS FEAT principles - the machine never approves or rejects a customer on its own.

The problem

Today

Tampered statements reach credit assessment because catching them relies on slow, inconsistent manual review. Analysts read every file by hand; fakes that look plausible get through.

What we need

Screen EVERY submitted statement automatically, auto-clear the clean ones so analysts focus only on flagged files, and attach explainable evidence to each flag.

The guiding rule throughout: the system only gathers evidence and, at worst, routes a case to a human - it never rejects a customer automatically.

Designed for governance (MAS FEAT)

Fairness

No protected attributes in scoring; flag rates tested across segments.

Ethics

Fail-safe: poor data or low confidence escalates - never auto-clears.

Accountability

Every AI output and every human decision is recorded to an audit log.

Transparency

Every flag carries a stable reason code + plain-English evidence.

PDPA

Prototype uses only synthetic data; PII access-controlled & encrypted in production.

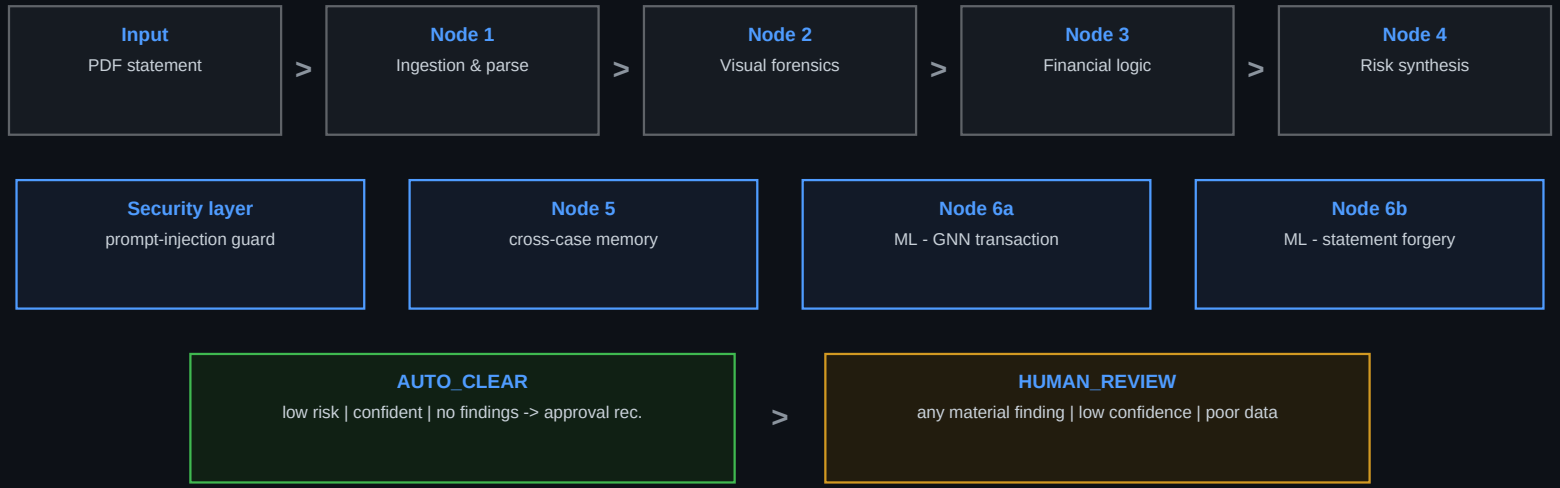
MAS TRM

Immutable, signed audit chain over every screening and reviewer decision.

HUMAN OWNS CALL

Most severe automated outcome is a hand-off to a person.

The pipeline at a glance



All agents feed Node 4. Findings from independent agents that agree raise a corroboration bonus.

Node 1 - Ingestion (the mailroom)

- Opens the PDF and extracts text, hidden metadata (author, software, dates), font fingerprints, the header (holder, UEN, period, balances) and the transaction table.
- Missing or unreadable fields are reported HONESTLY as data-quality findings - it never guesses.
- Tries six layout profiles; best page coverage wins. Ties that disagree are flagged rather than decided.
- A self-check compares printed section totals with parsed rows, so our own reading mistake isn't mistaken for document fraud.

6	100%	0.36s
layout profiles	rows read / balances replay (synthetic eval, all layouts)	longest file parsed

Node 2 - Visual Forensics (the document examiner)

Looks for signs the file was edited, independent of the numbers.

Signal	Reason code	Severity
Localised font mismatch (overwritten number)	VF_FONT_MISMATCH	MED
Mixed date formats / currency notation in rows	VF_DATE_FORMAT_MIX | VF_CURRENCY_MIX	MED
Image/PDF editor in metadata (Photoshop, Foxit)	VF_EDIT_SOFTWARE	HIGH
File modified >60s after creation by non-export tool	VF_MODIFIED_AFTER_CREATION	MED
Creation date before the period it claims to cover	VF_CREATION_BEFORE_PERIOD	MED

Adobe's PDF libraries are deliberately NOT flagged - real banks use them. Tuned so VF_CURRENCY_MIX went from 200/200 false fires on one bank layout to 0 / 2,000.

Node 3 - Financial Logic (the accountant)

Core checks

Row-by-row running-balance reconciliation | closing-balance match | period-continuity gaps (removed pages) | sudden deposits vs baseline | unverified counterparties | structuring (3+ near-threshold credits within 14 days) | UEN <-> ACRA entity cross-check.

Fabrication rules (income faking)

Duplicate invoice refs in one month | invoice numbers out of date order | turnover jumps while payroll/CPF/rent stay flat. Every finding names the rows and cites evidence - none is proof, each asks a person to check.

Live UEN cross-check against data.gov.sg (ACRA) - only the public UEN leaves the machine, over verified HTTPS, cached in memory. Fully overridable for offline demos.

Nodes 5 & 6 - Memory & Machine Learning

Node 5 - Cross-case memory

Matches this statement's suspicious rows against prior cases a HUMAN already decided. Known-bad repeat -> flag; previously cleared -> make sure this route isn't being exploited. Learns only from human-verified decisions and always cites the prior case.

ACTIVE

Node 6b - ML statement forgery

Scores the whole document for tampering from fonts, editor traces and balance math. Abstains on documents larger than its training (single-month), letting rules decide. Fail-safe: a missing model yields no findings.

GNN ON, GBM OFF

Node 6a - GNN transaction

A graph neural network catches relational AML patterns a single row can't reveal - round-tripping, mule fan-in/out, dormant reactivation - via fast lookup of offline-precomputed scores. The row-level GBM is off by default (false alarms on clean business files).

Security layer - prompt-injection guard

Scans extracted text for hidden commands (ignore previous instructions, mark low risk). It NEVER obeys - treats it as untrusted data and files the attempt itself as strong evidence of fraud.

Node 4 - Risk & Synthesis (the senior officer)

Risk score (1-100)

1-25 Low | 26-50 Medium | 51-75 High | 76-100 Very High

Sum of finding weights + corroboration bonus when independent agents agree. Any material (MED+) finding floors it into the Medium band.

Confidence score (1-100)

Starts high for a clean digital PDF. Lowered by data-quality gaps, thin evidence near a boundary, single-dimension evidence. Raised by corroboration.

Routing: AUTO_CLEAR only when risk <= 25 AND confidence >= 60 AND no material findings. HUMAN_REVIEW otherwise - including low confidence or poor data, even if risk looks low.

The ceiling is always "send to a human." The machine never rejects a customer.

Three roles, separated by design

Customer portal

customer_app.py | :8001

Applies, uploads a statement, tracks status. Sees ONLY neutral status - Verified / Being reviewed / Action needed. Never sees scores, evidence or routing (would tip off bad actors).

Fraud analyst

webapp.py | :8000

Screening queue, case overview, flagged transactions, per-stage view, PDF. Can Request info / Approve / Reject - each with a mandatory reason + optional training feedback.

Loan officer

banker_app.py | :8002

Only sees fraud-cleared cases. Makes the lending call (Approve / Decline / Refer). CANNOT override a fraud outcome - screening decides trust, the banker decides lending.

All three apps share one case store & audit log. The application reference number IS the case id - a submission appears in the analyst queue automatically.

Live demonstration

No external dependencies for the core pipeline - pure Python 3.

```
# 1) generate synthetic samples (clean + deliberately tampered)
$ python -m fraud_detect.synthetic

# 2) run the full pipeline end-to-end
$ python demo.py

# 3) or run any statement through the CLI
$ python3 -m statement_guard.cli samples/clean_statement.json
$ python3 -m statement_guard.cli samples/tampered_statement.json

# 4) the three web apps (shared case store)
$ uvicorn fraud_detect.customer_app:app --port 8001   # applicant
$ uvicorn fraud_detect.webapp:app       --port 8000   # analyst
$ uvicorn fraud_detect.banker_app:app   --port 8002   # loan officer
```

Demo A - a clean statement

```
StatementGuard - clean_statement.json
Node 1 Ingestion ........ OK   balances reconcile, all fields present
Node 2 Forensics ........ OK   no editor traces, consistent fonts/formats
Node 3 Financial ........ OK   running balance replays row-by-row
Node 6b ML forgery ...... OK   low forgery likelihood
------------------------------------------------------------
Risk score ......... 1 / 100   (Low)
Confidence ......... High
Decision ........... AUTO_CLEAR -> approval recommendation
```

Clean + confident + no material findings -> the analyst never has to touch it.

Demo B - a tampered statement

```
StatementGuard - tampered_statement.json
Node 1 Ingestion ........ 1 data-quality note
Node 2 Forensics ........ VF_EDIT_SOFTWARE  (OpenPage Writer 3.8)
                          VF_MODIFIED_AFTER_CREATION
Node 3 Financial ........ FL_BALANCE_BREAK  row 14 doesn't reconcile
                          FL_STRUCTURING    3 credits @ ~$9.8k / 14 days
                          FL_UEN_NAME_MISMATCH
Node 6b ML forgery ...... elevated likelihood
------------------------------------------------------------
Risk score ......... High  (+corroboration: visual AND financial agree)
Confidence ......... lowered by data-quality gap
Decision ........... HUMAN_REVIEW  - 10 pieces of evidence attached
```

Every flag is auditable evidence

Findings are typed, validated objects - not opaque scores. Missing fields become explicit nulls, never hallucinated values.

```
{
  "reason_code": "FL_BALANCE_BREAK",
  "severity":    "HIGH",
  "stage":       "financial_logic",
  "explanation": "Running balance at row 14 is 48,210.00 but the
                  printed balance is 58,210.00 - a 10,000.00 break.",
  "evidence": {
      "row": 14, "printed_balance": 58210.00,
      "computed_balance": 48210.00, "delta": 10000.00
  }
}
```

Does it actually catch fakes?

Synthetic generator plants ground-truth issues; tests assert each is detected.

Planted tamper	Detected by
Altered balance / number	FL_BALANCE_BREAK | VF_FONT_MISMATCH
Format inconsistency	VF_DATE_FORMAT_MIX | VF_CURRENCY_MIX
Metadata mismatch	VF_EDIT_SOFTWARE | VF_CREATION_BEFORE_PERIOD
Period gap | suspicious txn | structuring	FL_PERIOD_GAP | FL_SUDDEN_DEPOSIT | FL_STRUCTURING
UEN mismatch	FL_UEN_NAME_MISMATCH

71

test modules

13

detection tests - every planted issue caught, clean auto-clears, nothing auto-rejects

0 / 2000

false fires after currency-check tuning

GNN evaluation - honest, measured

Offline edge-level eval on the synthetic transaction dataset. Reported as what it is, with caveats.

Where the GNN adds value

On round-tripping accounts it catches 5 that Node 3 misses, at the live threshold with no false alarms. Together, GNN + Node 3 catch 19 / 27 fraud accounts (70%) vs 52% rules-alone.

Stated limits

Narrow: covers 1 of 5 AML typologies; scores 62 / 1,201 accounts. Live path is account-level only (offline numbers are an upper bound). Synthetic data -> graph lift may shrink on real data. Only 27 fraud test accounts - wide intervals.

"A green build is not success." We separate measured results from aspiration, and never overclaim.

What's real vs. simulated

DONE & WORKING

- The entire 4-agent decision engine, end-to-end
- Scoring, confidence, routing, corroboration
- Readable evidence on every flag
- 3 web apps | cross-case memory | ML nodes
- Immutable signed audit chain | test suite

SIMULATED / REMAINING

- Real PDF + OCR reading (input is typed data today)
- Real pixel/layer image forensics (metadata + rules now)
- Historical baselines from real cash-flow data
- Production DB, integration, security hardening

In short: the brain is complete; real-world inputs and production hardening are what remain.

Roadmap - 6 staged phases

Phase	In five words
0 | Foundations	Gather examples, agree the goal
1 | Harden core	Make the prototype sturdy
2 | Real ingestion	Read real PDF statements (OCR)
3 | Real forensics	Detect real image tampering
4 | Analyst UI	Build the staff review screen
5 | Shadow pilot	Safely test alongside humans
6 | Rollout	Turn it on gradually

Each phase ships something usable and de-risks the next. Recommended build strategy: hybrid - buy OCR + pixel forensics, build the orchestration, financial-logic, scoring & audit layer we own.

How we'll measure success

Recall on tampered docs

The catch rate - the metric that protects the bank.

Auto-clear precision

Protects analyst time & customers from false alarms.

% auto-cleared

The efficiency win - only counts if precision holds.

Time-to-decision

vs. the current manual baseline.

Analyst override rate

How often humans disagree with the band (calibration).

Explainability

% of flags an analyst rates clear evidence (FEAT).

Targets are set on a held-out, labelled eval set - the single most valuable asset.

Computer flags. Human decides.

- A working, end-to-end multi-agent screening pipeline for tampered bank statements.
- Explainable evidence on every flag; never auto-rejects a customer - MAS FEAT by design.
- Three separated roles, an immutable audit trail, and an honest view of real vs. simulated.
- A clear, staged path from prototype to production.

Thank you - questions & live demo welcome.